Tech.UB.27.002 Spring 2026 with Professor Guthrie Collin: Homework 4

Overall Description of Program being created across Homeworks 2, 3 and 4:

Your friend is concerned about protecting her friends and family information in her smartphone's address book. But she also wants the social graph information available when she links the address book to social media platforms. Her problem has inspired you to build a prototype "Social Address Book" program in Python. You will develop this prototype in three parts in homeworks 2, 3, and 4. In Homework 2, you will create the base Address Book program using Object Oriented Programming (14 points). In Homework 3, you will add sorting and searching functionality to the Address Book (8 points). In Homework 4, you will evolve the program into a Social Address Book by adding social graph functionality (8 points).

Specific Assignment for Homework 4:

The final step in your journey towards creating the Social Address Book is evolving your Address Book program to include a graph of your Contacts. Your program will enable an Address Book user to add and remove undirected edges between their Contacts. The program will also enable a user to display a single Contact's entire network within the Address Book up to the Contact's 3rd degree of connections. This display capability will be made possible by the implementation of Breadth First Search. These functions will be a prototype of a Social Address Book you want to show your friend. To do so, you will create a demonstration program of a Social Address Book with 20 individual Contacts. Each contact will have edges to two other Contacts. The demonstration program will be available in a *social_demo.py* file.

Code Requirements: Your program must conform to these requirements:

1. Program updates your Address Book to create a Social Address Book by including new attributes and methods to provide graph capabilities focused on Contact objects. New attributes and methods have appropriate documentation in the updated AddressBook class as described by the MEDIUM documentation.
2. The new functionality must be accompanied by a DEMONSTRATION PROGRAM stored in a new *social_demo.py* file that provides an illustrative example of the graph capabilities with at least 20 Contacts.
3. GRAPH STORAGE: every Contact has edges to two other Contacts. The collection of graph edges is stored as a new dictionary attribute in the AddressBook class. Edges are only created via a new method.
4. FIRST DEGREE GRAPH DISPLAY: the list of Contacts and their edges is displayable through a new AddressBook method that prints out each Contact and its neighbors (1st degree connections) to the terminal as a list (see example A below).
5. THIRD DEGREE GRAPH DISPLAY: the complete network up to the 3rd degree of edges for an individual Contact is displayable through a new AddressBook method that uses the Breadth First Search (BFS) algorithm to search all of the input Contact's connections up to the third degree and then prints this list out to the terminal by degree (see example B below).

Grading Rubric:

Point Value	Homework 4 Item
2	ANALYZE + DESIGN work quality (1 points) and program's use of KISS, DRY and SRP principles (1 point)
4	Fulfillment of Assignment through Code Requirements 1 to 4 (1 point per requirement)
2	Fulfillment of Assignment through Code Requirement 5
8	**TOTAL POINTS AVAILABLE FOR HOMEWORK 4**

(continued on next page)

EXAMPLE A: TERMINAL DISPLAY of a FIRST DEGREE GRAPH

```
--- NodeBook Adjacency List ---
Node  0 (Degree 3): connected to -> 1, 5, 19
Node  1 (Degree 2): connected to -> 0, 2
Node  2 (Degree 3): connected to -> 1, 3, 17
Node  3 (Degree 2): connected to -> 2, 4
Node  4 (Degree 2): connected to -> 3, 5
Node  5 (Degree 4): connected to -> 0, 4, 6, 10
Node  6 (Degree 2): connected to -> 5, 7
Node  7 (Degree 2): connected to -> 6, 8
Node  8 (Degree 2): connected to -> 7, 9
Node  9 (Degree 2): connected to -> 8, 10
Node 10 (Degree 4): connected to -> 5, 9, 11, 15
Node 11 (Degree 2): connected to -> 10, 12
Node 12 (Degree 2): connected to -> 11, 13
Node 13 (Degree 2): connected to -> 12, 14
Node 14 (Degree 2): connected to -> 13, 15
Node 15 (Degree 3): connected to -> 10, 14, 16
Node 16 (Degree 2): connected to -> 15, 17
Node 17 (Degree 3): connected to -> 2, 16, 18
Node 18 (Degree 2): connected to -> 17, 19
Node 19 (Degree 2): connected to -> 0, 18
```

EXAMPLE B: TERMINAL DISPLAY of an individual Node's THIRD DEGREE GRAPH

```
Enter Node ID (0-19) for BFS depth check: 15

--- 3rd Degree Connections for Node 15 ---
1st Degree     : [10, 14, 16]
2nd Degree     : [5, 9, 11, 13, 17]
3rd Degree     : [0, 2, 4, 6, 8, 12, 18]
```

IMPORTANT NOTE on use of ChatGPT, Gemini, or similar Large Language Models:

You are permitted to use an LLM tool for this homework assignment, however, you must cite your use of such tools in the completion of your assignment including a description of how you used those tools. Add this citation and detail to the TOP of your code file using comments.